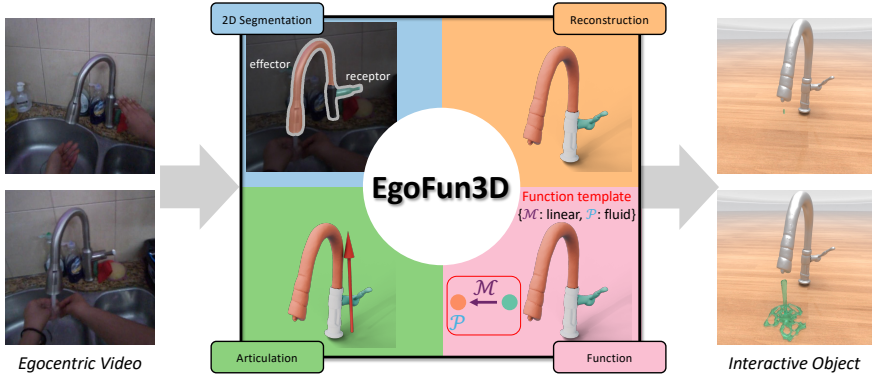


# EgoFun3D: Modeling Interactive Objects from Egocentric Videos using Function Templates

Weikun Peng, Denys Iliash, and Manolis Savva  
Simon Fraser University

[3dlg-hcvc.github.io/EgoFun3D/](https://3dlg-hcvc.github.io/EgoFun3D/)



**Fig. 1:** We present *EgoFun3D*: a coordinated task, dataset and benchmark for modeling interactive 3D objects from egocentric videos. Given an egocentric video as an input, the output is a simulation-ready interactive object (e.g., faucet handle starts water flow from faucet spout). We break down the task into 4 steps to propose a baseline approach using off-the-shelf components. Our *function template* representation, produced by the proposed system, can be compiled into executable code for a simulator of choice.

**Abstract.** We present *EgoFun3D*, a coordinated task formulation, dataset, and benchmark for modeling interactive 3D objects from egocentric videos. Interactive objects are of high interest for embodied AI but scarce, making modeling from readily available real-world videos valuable. Our task focuses on obtaining simulation-ready interactive 3D objects from egocentric video input. While prior work largely focuses on articulations, we capture general cross-part functional mappings (e.g., rotation of stove knob controls stove burner temperature) through *function templates*, a structured computational representation. Function templates enable precise evaluation and direct compilation into executable code across simulation platforms. To enable comprehensive benchmarking, we introduce a dataset of 271 *egocentric videos* featuring challenging real-world interactions with paired *3D geometry*, *segmentation over 2D and 3D*, *articulation* and *function template annotations*. To tackle the task, we propose a 4-stage pipeline consisting of: 2D part segmentation, reconstruction, articulation estimation, and function template inference. Comprehensive benchmarking shows that the task is challenging for off-the-shelf methods, highlighting avenues for future work.

**Keywords:** Interactive 3D objects, part functionality, egocentric video

# 1 Introduction

Objects composed of functional parts are *interactive*. Such objects have many applications in gaming, robotics and embodied AI [1, 12, 16, 22, 39, 44, 53, 61]. However, interactive 3D objects are scarcely available. Thus, modeling interactive objects from videos of real-world interactions is of increasing interest.

A special well-studied case of interactive objects is articulated objects, with a number of recent works investigating how to model part motion [23, 34, 55] as well as how to create articulated objects from real-world observations [33, 35, 36, 43, 56]. However, these works only model the movements of individual parts and not the interaction between parts and physical state changes. For instance, rotating the stove top knob will change the temperature of the stove burner.

While there are recent attempts at modeling such functionality [13, 60], they typically represent part functionality using natural language which is neither formal nor precise enough to enable structured evaluation or reliable conversion to simulation-ready assets. Other works [26] specify action state change using APIs provided by simulators. While these simulators allow implementing executable part functions [2, 28, 40], the APIs and code are heterogeneous and do not serve as a unified and portable representation of functionality.

To address this gap, we introduce a general notion of *part functionality* that maps how actions on one part (the *receptor*) lead to a response in another part (the *effector*).<sup>1</sup>

For example, pulling the faucet handle opens the faucet spout and achieves the intended response of water flowing as in Fig. 1. We propose *function templates*, a unified representation of part functionality that enables evaluation and compilation into code. The function template abstracts an object in terms of the receptor part that is manipulated and the effector part that produces a response (see Fig. 2). Function templates distill part functionality into two key components: *mapping* and *physical effect*. The mapping specifies how the receptor state influences the effector state. The physical effect specifies the physical phenomenon and units of the effector response.

Armed with our function template abstraction, we study the task of building interactive objects that can be used in simulation. As input, we focus on egocentric videos which are increasingly abundant and capture human-object activities with common interactive objects [18, 45]. We formulate and tackle the new task of modeling interactive objects using function templates from egocentric video data. This task poses many challenges, as egocentric videos feature *highly dynamic viewpoints, severe occlusions, and complex interactions*. As no prior dataset contains all the modalities required to benchmark this task (see Tab. 1), we create a benchmark dataset featuring: 1) real-world, egocentric videos; 2) 3D meshes of objects; 3) object and part segmentation annotations on both 2D and 3D; 4) part articulation annotations; and 5) function template annotations. The dataset and benchmarking suite will be made public.

---

<sup>1</sup> We take this terminology from biological reflex arcs [50] where receptors detect a stimulus and effectors produce an action in response.

**Table 1:** Dataset comparison. EgoFun3D provides real egocentric videos and corresponding geometry with the richest annotation compared to datasets from prior work. N.A. indicates annotation that cannot be present due to missing data modalities.

| Dataset                | Modality                        | Annotation    |               |              |                    |
|------------------------|---------------------------------|---------------|---------------|--------------|--------------------|
|                        |                                 | 2D Part Masks | 3D Part Masks | Articulation | Part Functionality |
| SceneFun3D [13]        | scanned point cloud             | N.A.          | ✓             | ✓            | ✓                  |
| FunGraph3D [60]        | scanned point cloud             | N.A.          | ✓             | ✗            | ✓                  |
| BEHAVIOR-1K [28]       | synthetic mesh                  | N.A.          | ✓             | ✓            | ✓                  |
| Articulate3D [20]      | synthetic mesh                  | N.A.          | ✓             | ✓            | ✗                  |
| PhysXNet [5]           | synthetic mesh                  | N.A.          | ✓             | ✓            | ✗                  |
| ArtVIP [24]            | synthetic mesh                  | N.A.          | ✓             | ✓            | ✓                  |
| Ego-Exo4D [18]         | real video                      | ✗             | N.A.          | N.A.         | ✗                  |
| HD-Epic [45]           | real video, synthetic mesh      | ✗             | ✗             | ✗            | ✗                  |
| Hoi! [15]              | real video, scanned point cloud | ✗             | ✓             | ✓            | ✗                  |
| Arti4D [57]            | real video, reconstructed mesh  | ✗             | ✗             | ✓            | ✗                  |
| iTACO [43]             | synthetic video, synthetic mesh | ✓             | ✓             | ✓            | ✗                  |
| <b>EgoFun3D (ours)</b> | real video, reconstructed mesh  | ✓             | ✓             | ✓            | ✓                  |

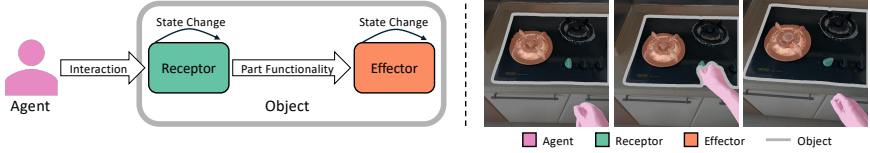
We propose a baseline approach for the task, powered by off-the-shelf components based on a 4-step breakdown: 1) segmenting the receptor and effector; 2) reconstructing the geometry of the receptor and effector; 3) inferring how the parts articulate; 4) inferring a suitable function template. We then use our dataset to perform extensive benchmarking. We find that objects with small, severely occluded parts in highly dynamic videos pose a challenge for every established off-the-shelf component used in our baseline approach.

In summary, our contributions include: 1) Formalizing the task of modeling interactive 3D objects from egocentric videos; 2) Introducing the function template representation to capture general part functionality and obtain simulator-ready objects; 3) Creating a dataset for comprehensive end-to-end benchmarking of off-the-shelf components on this task; 4) Proposing a baseline approach for the task using the best components identified by our benchmarking.

## 2 Related Work

**Articulated object modeling.** Articulated objects are a type of interactive object commonly seen in daily life. Modeling articulated objects has been studied for years. Most works focus on reconstructing interactive rigid or articulated objects from static scans [7–9, 29, 32, 33, 36, 38, 56, 59]. Recent works have begun using videos of manipulating articulated objects as input to build interactive articulated objects in simulation [27, 35, 43, 57]. These approaches take videos as input and output part-level reconstruction and articulation parameters. However, their focus is only on articulated parts, ignoring other functions such as illumination. Thus, these methods cannot be applied directly to our problem.

**Part functionality modeling.** Recent work has studied more general functional relationships between parts. SceneFun3D [13] introduces functionality segmentation and task-driven affordance grounding. One line of follow-up work studies how to better understand part functionality and affordance [11, 31]. Their



**Fig. 2:** Illustration of a typical form of human-object interaction. An agent interacts with a receptor, changing its state. *Part functionality* defines how the state change of the receptor maps to the state change of the effector. On the right, we provide an example of human interacting with a knob of the stove. The part function triggers the temperature change of the burner after knob actuation.

output is the segmentation of affordances on the point cloud. Another line of work attempts to build a functional 3D scene graph to represent the functional relationship between parts and objects [19, 49, 60]. MoMa-SG combines building 3D scene graphs and articulation estimation together to assist mobile manipulation [4]. This line of work is closer to our setting, but it takes a 3D scene understanding perspective, whereas we model interactive 3D objects. Moreover, they use natural language whereas we extract function templates that enable portable instantiation as executable functions across simulators.

**Datasets for part functionality.** At the scene level, SceneFun3D annotates interactive elements in 3D scenes with masks and natural language descriptions of their functions [13]. FunGraph3D annotates scene graphs to describe the functional relationship between different object parts [49]. Articulate3D annotates articulation parameters and part segmentation in 3D scenes [20]. These datasets focus on evaluating functionality understanding from a point cloud or a video of the static scene. At the object level, although several synthetic datasets either annotate part functionality in natural language [5] or implement executable part functionality in simulation [24, 28], they do not aim to evaluate modeling interactive objects with function templates from real videos. Arti4D [57] and iTACO [43] evaluate estimating articulated parameters from RGBD videos. Hoi [15] collects videos of manipulating articulated objects accompanied by force annotations and scanned point clouds of the objects. These datasets are the closest to our work, but they consider only rigid or articulated objects, whereas we model interactive objects with a broader notion of functionality.

### 3 Methodology

#### 3.1 Function Templates

Function templates are a concrete representation of part functionality. Our definition of part functionality is based on coupling the receptor and the effector (see Fig. 2): 1) An embodied agent actuates the receptor; 2) The receptor state change is mapped to the state change of the effector; 3) The effector state change causes further state changes in the environment. The goal of the agent is not to actuate the receptor, but to change the effector through actuating the receptor.

We denote the states of the receptor  $\mathcal{R}$  and effector  $\mathcal{E}$  by  $s_{\mathcal{E}}$  and  $s_{\mathcal{R}}$  respectively. Then, the part functionality is defined as  $s_{\mathcal{E}} = \mathcal{F}(s_{\mathcal{R}})$ . Function templates decompose the  $\mathcal{F}$  function into mapping  $\mathcal{M}$  and physical effect  $\mathcal{P}$ . Since the specifics of mapping (i.e., conversion of units) depend on physical effect, we instantiate part functionality via the function template as  $\mathcal{F}(\cdot) := \mathcal{M}_{\mathcal{P}}(\cdot)$ .

In egocentric videos of common indoor scene interaction scenarios, a few physical effects dominate that result in *visually-observable state changes*. These physical effects are changes of: 1) **geometry** (e.g. door opening); 2) **illumination** (e.g. light emission); 3) **temperature** (e.g. stove burner heating up); and 4) **fluid** (e.g. faucet spout releasing water). While conceptually  $\mathcal{P}$  can represent any physical effect, we target modeling these four as a foundation, as they are widely-present, supported by simulation platforms, and result in visually-observable changes.

Annotating mappings for each receptor and effector pair is challenging. First, accurate measurement of physical parameters such as illumination intensity, stove temperature, and water flow speed is impractical. Second, relevant parameters in current simulation platforms do not directly correspond to physical measurements and vary across simulators. For example, water flow speed for a faucet can be measured as the volume of water per second but it is typically parameterized by the number and speed of fluid particle [2].

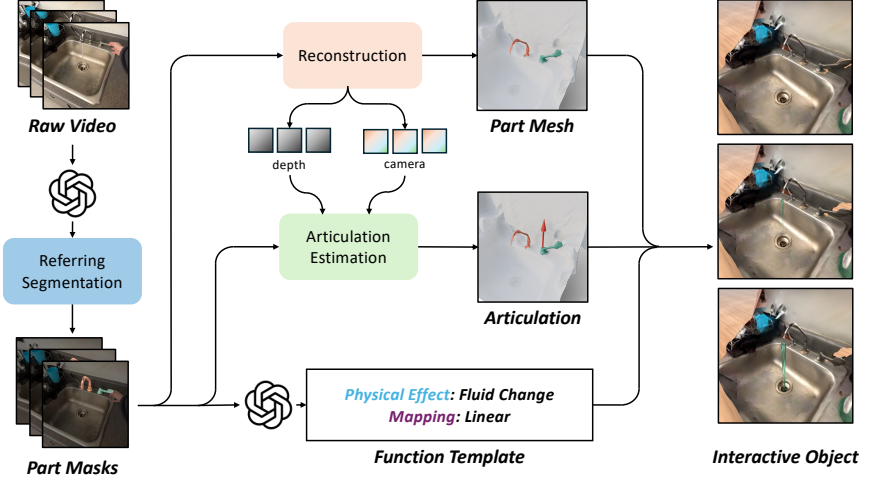
Due to these challenges, we abstract the mapping into a few types that strike a balance between generality and concreteness.

In practice, we observe 4 common types in our data which we represent using appropriate mathematical functions: 1) **binary** (e.g., button-type light switch turning on room light); 2) **step** (e.g., fridge door turning off fridge light at specific angle) 3) **linear** (e.g., knob-type light switch controlling lamp light intensity) 4) **cumulative** (e.g., electric stove with buttons controlling the increase or decrease of the burner’s temperature). See the supplement for a more detailed formalization.

The above formalization distills arbitrary real-world  $\mathcal{F}$  into two specific components  $\mathcal{P}$  and  $\mathcal{M}$  that can be modeled computationally. The resulting function templates are a unified representation enabling portability across simulators and easy evaluation. Arbitrary combinations of components in the function templates  $\mathcal{M}_{\mathcal{P}}$  can be compiled into executable Python scripts for specific simulator platforms. For example, binary and step functions can be represented using an if-else block, while linear and cumulative functions using simple linear equations with relevant physical parameters according to  $\mathcal{P}$  (see Fig. 4). In this paper, we show example outputs targeted to BEHAVIOR [28], Isaac Sim [40], and Genesis [2].

### 3.2 Task Definition

After formalizing function templates, we can now define the task. The input is an egocentric video  $\mathcal{V}$  capturing an agent interacting with an object. The output is an interactive object, parametrized by  $\{(\mathcal{G}_{\mathcal{R}}, \mathcal{A}_{\mathcal{R}}), (\mathcal{G}_{\mathcal{E}}, \mathcal{A}_{\mathcal{E}}), \mathcal{M}_{\mathcal{P}}\}$ . Where  $\mathcal{G}$  are per-part reconstructed 3D meshes.  $\mathcal{A}$  is an articulation parametrized by a tuple  $(J_t, J_a, J_o, J_r)$ , where  $J_t \in \{\text{fixed}, \text{prismatic}, \text{revolute}\}$  is a joint type,  $J_a \in$



**Fig. 3:** Our baseline framework. We break down the task into 4 steps that are individually targeted with off-the shelf components. First, a VLM generates part descriptions which are used to segment the parts in the video. Then, the geometry of the receptor and the effector are reconstructed, articulation parameters are estimated, and the function template is inferred. These outputs are combined to build the interactive object.

$\mathcal{R}^3$  is the joint axis,  $J_o \in \mathcal{R}^3$  is the joint origin and  $J_r \in \mathcal{R}^{2 \times 1}$  are the minimum and maximum values of range of motion.  $\mathcal{M}_{\mathcal{P}}$  is a function template as defined previously in Sec. 3.1, with  $\mathcal{M} \in \{\text{binary, step, linear, cumulative}\}$  and  $\mathcal{P} \in \{\text{geometry, illumination, temperature, fluid}\}$ .

### 3.3 Baseline Approach Framework

For our baseline approach, we propose to break the task down into 4 steps that can be implemented by off-the-shelf components: 2D segmentation, reconstruction, articulation estimation and function template inference. The proposed system is visualized in Fig. 3. First, we identify which parts are involved in the interaction and their functional roles. We prompt a VLM to identify the receptor and effector and describe them in detail. We then use these descriptions with a 2D referring segmentation model to obtain the part masks. We reconstruct the geometry of the receptor and effector by leveraging a reconstruction model combined with the predicted part masks. Such models also estimate the depth maps, camera intrinsics and extrinsics, which are used next by the articulation estimation model. We use a VLM once again to infer the mapping and physical effect, populating the function template. Finally, function templates are compiled into executable code and, along with part geometry and articulations, result in a simulation-ready interactive object.

## 4 Dataset

No prior dataset is suitable for evaluating our task end-to-end. Hence, we create a dataset to enable benchmarking. We describe how the data is curated (Sec. 4.1) and annotated (Sec. 4.2), and give summary statistics (Sec. 4.3).

### 4.1 Data Curation

Building a dataset for our task requires annotations across several modalities. However, existing datasets contain only egocentric videos or only 3D meshes. Thus, we curate available modalities from prior datasets, extending and annotating them to include the modalities needed for our task (see Fig. 4). We use three data sources: Ego-Exo4D [18], FunGraph3D [60], and self-captured data.

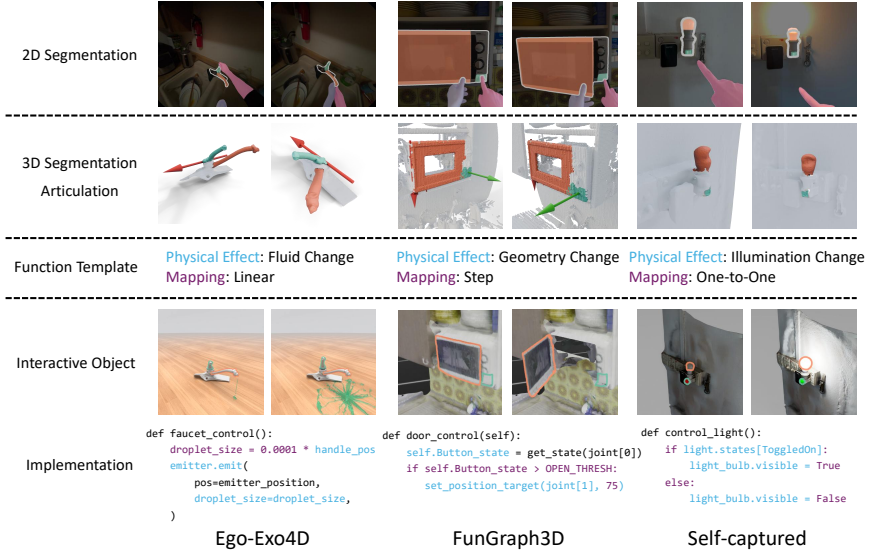
**Ego-Exo4D.** This dataset provides egocentric videos across 8 different scenarios [18]. Videos come with keysteps annotated, labeling the start and the end of each interaction episode. We identify a set of relevant keystone labels, and use them to cut original videos into short clips. Ego-Exo4D does not provide dense reconstructions of the indoor scenes or objects in 3D. Therefore, we use ShapeR [51] to generate object meshes from the provided egocentric videos and semi-dense point clouds. If ShapeR fails to generate an accurate mesh, we use MapAnything [25]. In such cases, we select a timestamp in the video and use both egocentric and exocentric views to build the object mesh.

**FunGraph3D.** This dataset provides egocentric videos in 14 indoor scenes and point clouds from a laser scanner. We first cut the videos into short clips manually such that each video clip involves manipulating one object. Since the videos do not come with intrinsics and camera poses, we manually mask out the moving regions in the videos and run MP-SfM [42] to estimate camera intrinsics and poses. FunGraph3D provides high-quality point clouds, so we refine the original point cloud segmentation and use it with the reconstructed object geometry.

**Self-captured.** To compensate for the imbalance of the distribution of object categories and function types in the previous two datasets, we also collect egocentric videos and object scans using an iPhone 11 Pro. We record interaction videos using Record3D [48], capturing RGBD videos, intrinsics, and camera poses. Then, we reconstruct 3D object meshes using Polycam [46].

### 4.2 Data Annotation

Our data annotation addresses 4 aspects: 2D segmentation masks for hand, object, and parts; 3D part segmentation masks; articulation parameters; and function templates. For 2D segmentation, we use SAM2 [47] to segment the objects and parts throughout the entire video clip. We annotate left and right hands, receptor and effector, and segmentation of the entire object. For 3D segmentation, we annotate receptor and effector masks on the reconstructed object meshes. For articulation parameters, we annotate joint type, joint axes, joint ranges and joint origins (for revolute joints). For function annotations, we build function templates by selecting appropriate mapping and physical effect pairs. Fig. 4 shows example data samples with annotations.



**Fig. 4:** Examples of annotations in our dataset. We provide 2D segmentation masks for hands, receptor (in teal), effector (in orange), and the whole object. We annotate part segmentation for receptor and effector on reconstructed 3D meshes. For articulation, we annotate revolute and prismatic joints, shown as red and green arrows respectively. For the function template, we pick one of four physical effects and one of four numerical expressions. Finally, we show concrete instantiations of interactive objects in different simulators: Genesis [2] (left), Isaac Sim [40] (middle), BEHAVIOR [28] (right).

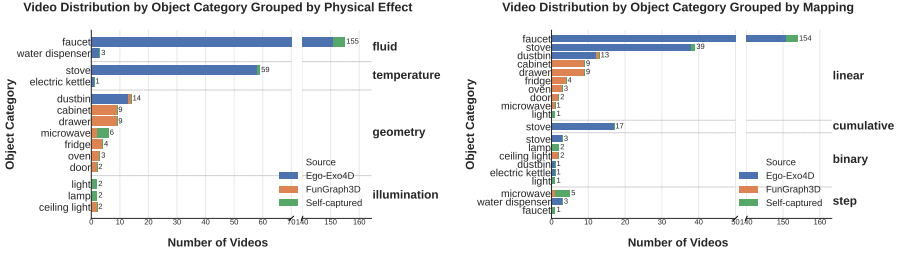
### 4.3 Dataset Statistics

In total, we collect 271 egocentric videos, featuring 88 different object instances across 14 different categories. Fig. 5 shows the data distributions. Data samples featuring faucets and stoves make up the majority of our dataset. This comes from the fact that we inherit the data distribution of our dominant data source, Ego-Exo4D. Ego-Exo4D features 8 scenarios (soccer, bike repair, cooking, etc.). Out of these, most videos featuring diverse functional objects in indoor environments with part functionalities that can be modeled in modern simulators come from the cooking scenario. This scenario is among the most functionally-complex common human activities except for interactions with computing devices. The data bias can also lead to a skewed distribution for functional mappings and physical effects as faucets and stoves are naturally biased towards certain combinations.

## 5 Experiments

We benchmark off-the-shelf components that implement the 4 steps of our task breakdown. In this section, we discuss the methods we benchmark for each step (Sec. 5.1), the evaluation protocol (Sec. 5.2) and our findings (Sec. 5.3).





**Fig. 5:** Our egocentric video dataset distributions across object categories. There are prominent long tail distributions across categories, physical effects, and function mappings, primarily due to inherited biases from source datasets such as Ego-Exo4D.

## 5.1 Baselines and Implementation Details

**2D segmentation.** We first use Gemini 3 Flash [17] to output a short label and a long description of the receptor and effector in the video clip. The long description provides spatial relationships (e.g., the *right-most* knob) to disambiguate the instance with which the human interacts in the video, as multiple parts with the same label might be present. See the supplement for the VLM prompting details. We then select a few representative methods for referring expression segmentation. There are 2 categories of methods: those natively supporting text prompts, and those requiring geometric prompts (i.e., clicks, bounding boxes). X-SAM [54], Sa2VA [58] and SAM3 [6] are representative approaches that support text prompts natively. SAM3, however, is not meant to take long descriptions as input. Hence, we pair it with Qwen3-VL 8B [3] as per the SAM3-Agent setup in the public SAM3 codebase. This setup establishes an agentic framework where the VLM uses SAM3 as a tool. We also include a few models that output geometric prompts based on natural language descriptions, such as Molmo2 [10] and VisionReasoner [37] and combine them with SAM for segmentation. To achieve feasible runtimes, we subsample 20 frames from each video uniformly and segment them. Then, we use SAM3 to propagate the segmentation to all video frames, with the initial 20 acting as prompts. See the supplement for more details.

**Reconstruction.** We select three representative methods for reconstruction: MapAnything [25], Depth Anything 3 [30], and ViPE [21]. MapAnything and Depth Anything 3 are multi-view transformers that reconstruct 3D scenes from a series of images. ViPE is a more complex, multi-stage optimization framework. For MapAnything and Depth Anything 3, we divide the full video into several chunks with 20 frames for each chunk. Then, we reconstruct each chunk of the video individually and stitch the results of all chunks for computational feasibility. ViPE handles frame downsampling, so we send the full video once. The outputs require a post-processing step to fuse the point clouds into a single state, as the input videos feature moving parts. We use RoMa [14] for pair-wise feature matching to estimate rigid transformations between the frames and fuse the point cloud to the state of the first frame. Finally, we aggregate reconstructed

meshes for each part across all frames to obtain the final reconstruction. See the supplement for more details.

**Articulation estimation.** ArtiPoint [57] and iTACO [43] are two recent methods for articulation estimation. Artipoint uses point tracks to build a factor graph and estimate articulation parameters. We remove hand detection and object segmentation modules from the original pipeline, as segmentation masks in our setting are provided from an upstream prediction. iTACO first estimates the articulation parameters using image feature matching and then refines the initial proposal using a gradient-based optimization framework. We remove camera pose and moving part estimation modules from iTACO as those are provided from the upstream modules.

**Function template inference.** We compare four VLMs for function template estimation: Gemini 3 Flash [17], GPT-5 mini [41], Molmo2 8B [10], and Qwen3-VL 8B [3]. A video with receptor and effector parts highlighted in different colors is the input and we prompt VLM to infer 1 of 4 physical effects and 1 of 4 mappings.

## 5.2 Evaluation Protocol

We use IoU to evaluate the segmentation. The results are reported on masks propagated to all frames. As downstream pipeline steps rely on segmentation, we select only the parts with average IoU greater than 50% for further evaluation of the downstream modules. We consider such cases to be a segmentation success, and report the success rates additionally. The evaluation of downstream modules takes the successful masks predicted by the best method as input. This is done to decouple the effects of severely incorrect segmentation results from the raw performance of downstream modules while evaluating the performance of the entire pipeline considering reasonable error accumulation.

Reconstruction is evaluated using the chamfer distance for the receptor and effector. We report the median of the distance distribution of each part type separately, as well as across both part types combined. The choice of a median, in contrast to the mean, is due to occasionally-produced severely corrupted predictions that are capable of skewing the mean. See supplement for more discussion. We also report the mean value of the camera pose prediction error to assist analysis.

We use depth maps, camera intrinsics and extrinsics predicted by the best reconstruction method we identify, as input to articulation estimation models. Articulation estimation is evaluated with the joint type accuracy, joint axis error in radians, joint origin error in meters (for revolute joints only). Since the methods we use are capable of producing no output, we count a number of such occasions and report their fraction as a failure rate.

Since inference of function templates is formulated as a classification problem, we report accuracies for classifying physical effect and mapping, as well as an accuracy of predicting both at the same time.

We use Nvidia L40S GPUs to run our experiments and report runtimes. Evaluation of each module in oracle settings can be found in supplement.

**Table 2:** Evaluation of 2D segmentation performance. We find that SAM3 & Qwen3-VL outperforms other methods by a large margin, but is very inefficient.

| Model           | IoU (%)     |             |             | Success (%) |             | Avg. Run. (s) |
|-----------------|-------------|-------------|-------------|-------------|-------------|---------------|
|                 | Receptor    | Effector    | Avg.        | Receptor    | Effector    |               |
| VisionReasoner  | 14.6        | 34.9        | 24.7        | 7.4         | 33.9        | 452           |
| SAM3 & Qwen3-VL | <b>30.0</b> | <b>47.9</b> | <b>38.8</b> | <b>23.2</b> | <b>55.0</b> | 2012          |
| SAM3 & Molmo2   | 14.0        | 29.4        | 21.7        | 6.6         | 20.7        | 392           |
| X-SAM           | 2.5         | 15.0        | 8.7         | 0.4         | 9.2         | 255           |
| Sa2VA           | 14.8        | 43.4        | 29.1        | 1.8         | 33.2        | 2006          |



**Fig. 6:** Example 2D segmentation results. We find that SAM3 with Qwen3-VL provides the best segmentation. The main challenges in this subtask are segmentation of incorrect parts (left) and confusion between part instances across frames (middle). Performance on videos featuring more static viewpoints and no part instance ambiguity is better, though such videos are rare (right).

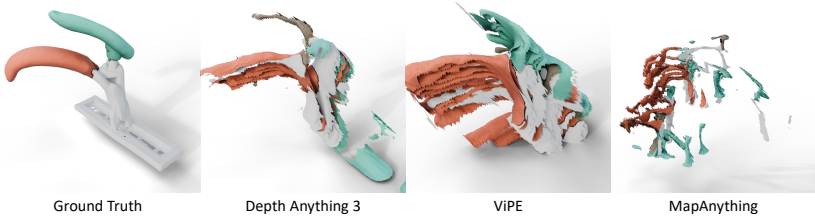
### 5.3 Experimental Results

**Segmentation consistency and small parts are bottlenecks.** Overall, the segmentation performance is unsatisfactory. We find that using SAM3 with Qwen3-VL [3] is the best option by a large margin. Hence, its outputs are used as inputs when evaluating the downstream modules. VisionReasoner strikes a better balance between performance and runtime. In general, segmenting receptors is much more challenging than segmenting effectors, as receptors tend to be smaller parts such as stove knobs and faucet handles. Fig. 6 shows qualitative examples and the supplement provides quantitative comparisons. We see that common failure modes include segmentation of incorrect parts and inconsistent segmentation of different instances across frames. The results suggest that the agentic framework for 2D segmentation with extensive reasoning is more promising, particularly for challenging scenarios.

**Reconstruction from egocentric videos is misaligned and incomplete.** Tab. 3 shows that the performance of MapAnything is the worst by a large margin. This happens due to severe problems with predicted camera intrinsics and extrinsics (see supplement for discussion). We hypothesize that the large performance gap between MapAnything and Depth Anything 3 is due to Depth

**Table 3:** Evaluating reconstruction. We report the median value of the chamfer distance and mean value of camera pose prediction error. Depth Anything 3 performs the best out of the methods we benchmark. MapAnything severely underperforms due to the camera predictions errors.

| Method           | Receptor CD (m <sup>2</sup> ) ↓ | Effector CD (m <sup>2</sup> ) ↓ | Total CD (m <sup>2</sup> ) ↓ | Camera Rot. Err. (rad) ↓ | Camera Tr. Err. (m) ↓ |
|------------------|---------------------------------|---------------------------------|------------------------------|--------------------------|-----------------------|
| MapAnything      | 0.380                           | 0.953                           | 0.580                        | 1.033                    | 0.742                 |
| Depth Anything 3 | <b>0.026</b>                    | <b>0.014</b>                    | <b>0.016</b>                 | <b>0.045</b>             | <b>0.049</b>          |
| ViPE             | 0.034                           | 0.021                           | 0.025                        | 0.046                    | 0.058                 |



**Fig. 7:** Example results for reconstruction. MapAnything exhibits severe drifting issues as predicted camera poses for different video frames are inaccurate. Other approaches also exhibit significant artifacts. Overall, reconstruction from our egocentric video data is highly challenging for all methods.

Anything 3 being trained on much more data, which covers a more diverse set of scenarios. Depth Anything 3 outperforms ViPE slightly and is therefore chosen as the method providing the inputs for the downstream. From Fig. 7, we identify two challenges of reconstruction. First, the reconstructed meshes across video frames are misaligned. Second, the meshes are incomplete even though we aggregate results across all frames. The fusion module is ineffective at aligning reconstructed meshes across frames. This is likely due to two reasons: 1) small and severely occluded parts do not provide sufficiently reliable feature matches to estimate transformations; and 2) parts that are highly reflective or have featureless textures make finding reliable matches even more difficult. As a result, we hypothesize that end-to-end, learned 4D reconstruction methods would better fit the task setting, as they may excel at tracking the points and aligning observations into a single state. 3D generative models are another potential solution to generate complete meshes, addressing the incompleteness issue.

**Tracking-based articulation prediction is unreliable.** From Tab. 4 and Fig. 8 we see that Artipoint generally outperforms iTACO, but suffers from much higher failure rates. As Artipoint relies on point tracking, it is not surprising that it fails frequently in our setting. Severe occlusions, small parts, and highly dynamic videos, all contribute to deteriorating the quality of point tracks. Consequently, we believe that if a method can estimate articulation parameters from hand motion rather than part motion, it can overcome this challenge, since hands are usually visible in egocentric videos and hand tracking is mature.

**VLMs are good at inferring function templates.**

**Table 4:** Evaluating articulation parameters estimation. We report the mean error across the videos that successfully go through the whole pipeline. We find that Artipoint is more accurate than iTACO, but is less robust. The overall performance for both methods is very low, indicating that articulation estimation is one of the bottlenecks.

| Method    | Joint Axis Err. (rad) ↓ | Joint Origin Err. (m) ↓ | Joint Type Acc. (%) ↑ | Failure Rate (%) ↓ |
|-----------|-------------------------|-------------------------|-----------------------|--------------------|
| ArtiPoint | 1.057                   | <b>0.346</b>            | <b>74.2</b>           | 46.4               |
| iTACO     | <b>1.022</b>            | 0.665                   | 26.8                  | <b>5.6</b>         |



**Fig. 8:** Example results for articulation estimation. Red arrows refer to revolute joints and green arrows refer to prismatic joints. In the left example, iTACO predicts incorrect joint types, whereas Artipoint is correct. However, both methods struggle with small parts such as the stove knob shown here.

Tab. 5 shows that Gemini 3 Flash is the most accurate model for this step, GPT-5 mini is the second best, while two open-source models underperform. Nonetheless, Qwen3-VL achieves somewhat satisfactory performance. Overall, VLMs are good at inferring function templates. We believe this is a strong signal towards the suitability of the proposed function template representation for the computational modeling of part functionality.

## 5.4 Final Outputs in Simulation

After predicting the function template, we can compile the template to specific simulators. Thus, our representation allows us to create interactive objects that can be used across simulators. See the supplement for a discussion of template compilation.

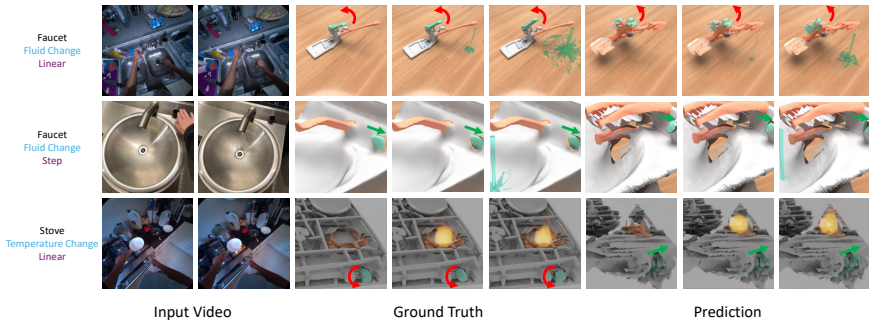
We demonstrate several results in Fig. 9. Overall, the issues described in Sec. 5.3 are also reflected here. In the second row, we see that meshes reconstructed from different video frames do not align well due to errors in camera pose estimation and depth estimation. In the third row, we see that articulation estimation is also challenging, particularly for small parts, resulting in the joint type for the stove knob being misclassified as prismatic by iTACO.

## 6 Conclusion

In this work, we proposed *EgoFun3D* a coordinated task, dataset and benchmark for modeling interactive 3D objects from egocentric videos. To tackle this

**Table 5:** Evaluation of function template inference accuracy. We report prediction accuracy for physical effect, mapping, and overall accuracy. A function template is correct if both effect and mapping are correct. We only report accuracy across videos where both receptor and effector segmentation IoUs are larger than 0.5. Among the four different VLMs we benchmarked on this task, Gemini-3-flash performs the best.

| Method         | Physical Effect Acc. (%) $\uparrow$ | Mapping Acc. (%) $\uparrow$ | Overall Acc. (%) $\uparrow$ |
|----------------|-------------------------------------|-----------------------------|-----------------------------|
| Gemini 3 Flash | 95.2                                | <b>97.6</b>                 | <b>92.9</b>                 |
| GPT-5 mini     | 88.1                                | 90.5                        | 83.3                        |
| Molmo2 8B      | 90.5                                | 31.0                        | 28.6                        |
| Qwen3-VL 8B    | <b>97.6</b>                         | 76.2                        | 76.2                        |



**Fig. 9:** Qualitative results of the final outputs of our system. The first two rows illustrate two interactive faucets in Genesis [2]. The last row illustrates an interactive stove in BEHAVIOR-1K [28]. We use teal to indicate receptors and orange to indicate effectors. Red and green arrows represent revolute and prismatic joints respectively.

task, we first presented a function template representation that accurately captures the mapping of cross-part functionality and resulting physical effects. We showed how this representation enabled portability across simulation platforms through compilation into executable code functions. We then proposed a system to decompose the overall task into four subtasks: 2D part segmentation, reconstruction, articulation estimation and function prediction. To evaluate our system comprehensively, we collected and annotated a dataset that provides annotations across egocentric videos and reconstructed 3D objects. We benchmarked off-the-shelf components implementing the system on EgoFun3D. Our results showed that 2D segmentation of small parts, reconstruction from dynamic videos, and reliable articulation estimation are open challenges. Better and more efficient agentic 2D segmentation frameworks, more accurate end-to-end 4D reconstruction, and more robust articulation estimation based on hand motion are promising directions for future work. Moreover, our work was limited to a subset of common human activities and part functionalities. Expanding the dataset and the scope of part functionality understanding for interactive 3D object modeling is another exciting avenue for future work.

## Acknowledgments

This work was funded in part by a Canada Research Chair, NSERC Discovery Grant, and enabled by support from the Digital Research Alliance of Canada. The authors would like to thank Tianrun Hu from National University of Singapore for collecting data, Jiayi Liu, Xingguang Yan, Austin T. Wang, and Morteza Badali for valuable discussions and proofreading.

## References

1. Abou-Chakra, J., Rana, K., Dayoub, F., Suenderhauf, N.: Physically Embodied Gaussian Splatting: A Realtime Correctable World Model for Robotics. In: 8th Annual Conference on Robot Learning (2024) 2
2. Authors, G.: Genesis: A Generative and Universal Physics Engine for Robotics and Beyond (December 2024), <https://github.com/Genesis-Embodied-AI/Genesis> 2, 5, 8, 14, 20, 21, 22, 29
3. Bai, S., Cai, Y., Chen, R., Chen, K., Chen, X., Cheng, Z., Deng, L., Ding, W., Gao, C., Ge, C., Ge, W., Guo, Z., Huang, Q., Huang, J., Huang, F., Hui, B., Jiang, S., Li, Z., Li, M., Li, M., Li, K., Lin, Z., Lin, J., Liu, X., Liu, J., Liu, C., Liu, Y., Liu, D., Liu, S., Lu, D., Luo, R., Lv, C., Men, R., Meng, L., Ren, X., Ren, X., Song, S., Sun, Y., Tang, J., Tu, J., Wan, J., Wang, P., Wang, P., Wang, Q., Wang, Y., Xie, T., Xu, Y., Xu, H., Xu, J., Yang, Z., Yang, M., Yang, J., Yang, A., Yu, B., Zhang, F., Zhang, H., Zhang, X., Zheng, B., Zhong, H., Zhou, J., Zhou, F., Zhou, J., Zhu, Y., Zhu, K.: Qwen3-vl technical report. arXiv preprint arXiv:2511.21631 (2025) 9, 10, 11
4. Buechner, M., Roefer, A., Engelbracht, T., Welschehold, T., Bauer, Z., Blum, H., Pollefeys, M., Valada, A.: Articulated 3D Scene Graphs for Open-World Mobile Manipulation. arXiv preprint arXiv:2602.16356 (2026) 4
5. Cao, Z., Chen, Z., Pan, L., Liu, Z.: PhysX-3D: Physical-Grounded 3D Asset Generation. In: NeurIPS (2025) 3, 4
6. Carion, N., Gustafson, L., Hu, Y.T., Debnath, S., Hu, R., Suris, D., Ryali, C., Alwala, K.V., Khedr, H., Huang, A., Lei, J., Ma, T., Guo, B., Kalla, A., Marks, M., Greer, J., Wang, M., Sun, P., Rädle, R., Afouras, T., Mavroudi, E., Xu, K., Wu, T.H., Zhou, Y., Momeni, L., Hazra, R., Ding, S., Vaze, S., Porcher, F., Li, F., Li, S., Kamath, A., Cheng, H.K., Dollár, P., Ravi, N., Saenko, K., Zhang, P., Feichtenhofer, C.: SAM 3: Segment Anything with Concepts (2025) 9
7. Chen, C., Liu, I., Wei, X., Su, H., Liu, M.: FreeArt3D: Training-Free Articulated Object Generation using 3D Diffusion. In: Proceedings of the SIGGRAPH Asia 2025 Conference Papers (2025) 3
8. Chen, H., Lan, Y., Chen, Y., Pan, X.: ArtiLatent: Realistic Articulated 3D Object Generation via Structured Latents. In: Proceedings of the SIGGRAPH Asia 2025 Conference Papers (2025) 3
9. Chen, Z., Walsman, A., Memmel, M., Mo, K., Fang, A., Vemuri, K., Wu, A., Fox, D., Gupta, A.: URDFormer: A Pipeline for Constructing Articulated Simulation Environments from Real-World Images. In: Robotics: Science and Systems (RSS) (2024) 3
10. Clark, C., Zhang, J., Ma, Z., Park, J.S., Salehi, M., Tripathi, R., Lee, S., Ren, Z., Kim, C.D., Yang, Y., Shao, V., Yang, Y., Huang, W., Gao, Z., Anderson, T., Zhang,

- J., Jain, J., Stoica, G., Han, W., Farhadi, A., Krishna, R.: Molmo2: Open Weights and Data for Vision-Language Models with Video Understanding and Grounding. *arXiv* (2026) 9, 10
11. Corsetti, J., Giuliani, F., Fasoli, A., Boscaini, D., Poiesi, F.: Functionality understanding and segmentation in 3D scenes. In: *Proceedings of the Computer Vision and Pattern Recognition Conference*. pp. 24550–24559 (2025) 3
  12. Dan, P., Kedia, K., Chao, A., Duan, E.W., Pace, M.A., Ma, W.C., Choudhury, S.: X-Sim: Cross-Embodiment Learning via Real-to-Sim-to-Real. *CoRL* (2025) 2
  13. Delitzas, A., Takmaz, A., Tombari, F., Sumner, R., Pollefeys, M., Engelmann, F.: SceneFun3D: Fine-Grained Functionality and Affordance Understanding in 3D Scenes. In: *CVPR* (2024) 2, 3, 4
  14. Edstedt, J., Sun, Q., Bökman, G., Wadenbäck, M., Felsberg, M.: RoMa: Robust Dense Feature Matching. In: *IEEE Conference on Computer Vision and Pattern Recognition* (2024) 9
  15. Engelbracht, T., Zurbrügg, R., Wohlrapp, M., Büchner, M., Valada, A., Pollefeys, M., Blum, H., Bauer, Z.: Hoi!-A Multimodal Dataset for Force-Grounded, Cross-View Articulated Manipulation. In: *CVPR* (2026) 3, 4
  16. Escontrela, A., Kerr, J., Allshire, A., Frey, J., Duan, R., Sferrazza, C., Abbeel, P.: GaussGym: An Open-Source Real-To-Sim Framework for Learning Locomotion from Pixels. *CoRR* (2025) 2
  17. Gemini Team: A new era of intelligence with Gemini 3 (2025), <https://blog.google/products-and-platforms/products/gemini/gemini-3/> 9, 10, 23
  18. Grauman, K., Westbury, A., Torresani, L., Kitani, K., Malik, J., Afouras, T., Ashutosh, K., Baiyya, V., Bansal, S., Boote, B., et al.: Ego-Exo4D: Understanding Skilled Human Activity from First- and Third-Person Perspectives. In: *CVPR* (2024) 2, 3, 7
  19. Gu, Q., Sheng, Y., Yu, J., Tang, J., Shan, X., Shen, Z., Yi, T., Liang, X., Chen, X., Wang, Y.: ArtiSG: Functional 3D Scene Graph Construction via Human-demonstrated Articulated Objects Manipulation. *arXiv preprint arXiv:2512.24845* (2025) 4
  20. Halacheva, A.M., Miao, Y., Zaech, J.N., Wang, X., Van Gool, L., Paudel, D.P.: Articulate3D: Holistic Understanding of 3D Scenes as Universal Scene Description. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* (2025) 3, 4
  21. Huang, J., Zhou, Q., Rabeti, H., Korovko, A., Ling, H., Ren, X., Shen, T., Gao, J., Slepichev, D., Lin, C.H., Ren, J., Xie, K., Biswas, J., Leal-Taixe, L., Fidler, S.: ViPE: Video Pose Engine for 3D Geometric Perception. In: *NVIDIA Research Whitepapers arXiv:2508.10934* (2025) 9
  22. Jiang, H., Hsu, H.Y., Zhang, K., Yu, H.N., Wang, S., Li, Y.: Phystwin: Physics-informed reconstruction and simulation of deformable objects from videos. *ICCV* (2025) 2
  23. Jiang, H., Mao, Y., Savva, M., Chang, A.X.: OPD: Single-view 3D openable part detection. In: *European Conference on Computer Vision*. pp. 410–426. Springer (2022) 2
  24. Jin, Z., Che, Z., Zhao, Z., Wu, K., Zhang, Y., Zhao, Y., Liu, Z., Zhang, Q., Ju, X., Tian, J., et al.: ArtVIP: Articulated Digital Assets of Visual Realism, Modular Interaction, and Physical Fidelity for Robot Learning. In: *ICLR* (2026) 3, 4
  25. Keetha, N., Müller, N., Schönberger, J., Porzi, L., Zhang, Y., Fischer, T., Knapitsch, A., Zauss, D., Weber, E., Antunes, N., Luiten, J., Lopez-Antequera,



- M., Bulò, S.R., Richardt, C., Ramanan, D., Scherer, S., Kotschieder, P.: MapAnything: Universal Feed-Forward Metric 3D Reconstruction. In: International Conference on 3D Vision (2026) 7, 9, 24
26. Kolve, E., Mottaghi, R., Han, W., VanderBilt, E., Weihs, L., Herrasti, A., Deitke, M., Ehsani, K., Gordon, D., Zhu, Y., et al.: AI2-THOR: An interactive 3D environment for visual AI. arXiv preprint arXiv:1712.05474 (2017) 2
  27. Le, L., Xie, J., Liang, W., Wang, H.J., Yang, Y., Ma, Y.J., Vedder, K., Krishna, A., Jayaraman, D., Eaton, E.: Articulate-Anything: Automatic Modeling of Articulated Objects via a Vision-Language Foundation Model. In: International Conference on Learning Representations (ICLR) (2025) 3
  28. Li, C., Zhang, R., Wong, J., Gokmen, C., Srivastava, S., Martín-Martín, R., Wang, C., Levine, G., Lingelbach, M., Sun, J., et al.: BEHAVIOR-1K: A Human-Centered, Embodied AI Benchmark with 1,000 Everyday Activities and Realistic Simulation. In: Conference on Robot Learning. pp. 80–93. PMLR (2023) 2, 3, 4, 5, 8, 14, 21, 22, 29
  29. Li, Z., Zhang, C., Li, Z., Howard-Jenkins, H., Lv, Z., Geng, C., Wu, J., Newcombe, R., Engel, J., Dong, Z.: ART: Articulated Reconstruction Transformer. In: CVPR (2025) 3
  30. Lin, H., Chen, S., Liew, J.H., Chen, D.Y., Li, Z., Shi, G., Feng, J., Kang, B.: Depth Anything 3: Recovering the Visual Space from Any Views. arXiv preprint arXiv:2511.10647 (2025) 9, 24
  31. Liu, C., Zhai, W., Yang, Y., Luo, H., Liang, S., Cao, Y., Zha, Z.J.: Grounding 3D Scene Affordance From Egocentric Interactions. arXiv preprint arXiv:2409.19650 (2024) 3
  32. Liu, J., Iliash, D., Chang, A.X., Savva, M., Mahdavi-Amiri, A.: SINGAPO: Single Image Controlled Generation of Articulated Parts in Object. In: International Conference on Learning Representations (ICLR) (2025) 3
  33. Liu, J., Mahdavi-Amiri, A., Savva, M.: PARIS: Part-level Reconstruction and Motion Analysis for Articulated Objects. In: Proceedings of the IEEE International Conference on Computer Vision (ICCV) (2023) 2, 3
  34. Liu, J., Savva, M., Mahdavi-Amiri, A.: Survey on Modeling of Human-made Articulated Objects. In: Computer Graphics Forum. vol. 44, p. e70092. Wiley Online Library (2025) 2
  35. Liu, Y., Jia, B., Lu, R., Gan, C., Chen, H., Ni, J., Zhu, S.C., Huang, S.: VideoArtGS: Building Digital Twins of Articulated Objects from Monocular Video. arXiv preprint arXiv:2509.17647 (2025) 2, 3
  36. Liu, Y., Jia, B., Lu, R., Ni, J., Zhu, S.C., Huang, S.: Building Interactable Replicas of Complex Articulated Objects via Gaussian Splatting. In: International Conference on Learning Representations (ICLR) (2025) 2, 3
  37. Liu, Y., Qu, T., Zhong, Z., PENG, B., Liu, S., Yu, B., Jia, J.: VisionReasoner: Unified Reasoning-Integrated Visual Perception via Reinforcement Learning. In: ICLR (2026) 9
  38. Mandi, Z., Weng, Y., Bauer, D., Song, S.: Real2Code: Reconstruct Articulated Objects via Code Generation. In: International Conference on Learning Representations (ICLR) (2025) 3
  39. Ning, C., Fang, K., Ma, W.C.: Prompting with the Future: Open-World Model Predictive Control with Interactive Digital Twins. In: RSS (2025) 2
  40. NVIDIA: Isaac Sim, <https://github.com/isaac-sim/IsaacSim> 2, 5, 8, 20, 21, 29
  41. OpenAI: Introducing GPT-5. <https://openai.com/index/introducing-gpt-5/> (2025) 10

42. Pataki, Z., Sarlin, P.E., Schönberger, J.L., Pollefeys, M.: MP-SfM: Monocular Surface Priors for Robust Structure-from-Motion. In: CVPR (2025) 7
43. Peng, W., Lv, J., Lu, C., Savva, M.: iTACO: Interactable Digital Twins of Articulated Objects from Casually Captured RGBD Videos. In: International Conference on 3D Vision (2026) 2, 3, 4, 10
44. Peng, W., Lv, J., Zeng, Y., Chen, H., Zhao, S., Sun, J., Lu, C., Shao, L.: TieBot: Learning to Knot a Tie from Visual Demonstration through a Real-to-Sim-to-Real Approach. In: 8th Annual Conference on Robot Learning (2024) 2
45. Perrett, T., Darkhalil, A., Sinha, S., Emara, O., Pollard, S., Parida, K., Liu, K., Gatti, P., Bansal, S., Flanagan, K., Chalk, J., Zhu, Z., Guerrier, R., Abdelazim, F., Zhu, B., Moltisanti, D., Wray, M., Doughty, H., Damen, D.: HD-EPIC: A Highly-Detailed Egocentric Video Dataset. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) (June 2025) 2, 3
46. Polycam: Polycam (2025), <https://poly.cam/> 7
47. Ravi, N., Gabeur, V., Hu, Y.T., Hu, R., Ryali, C., Ma, T., Khedr, H., Rädle, R., Rolland, C., Gustafson, L., Mintun, E., Pan, J., Alwala, K.V., Carion, N., Wu, C.Y., Girshick, R., Dollár, P., Feichtenhofer, C.: SAM 2: Segment Anything in Images and Videos. arXiv preprint arXiv:2408.00714 (2024) 7
48. Record3D: Record3d (2025), <https://record3d.app/> 7
49. Rotondi, D., Scaparro, F., Blum, H., Arras, K.O.: FunGraph: Functionality Aware 3D Scene Graphs for Language-Prompted Scene Interaction. In: IEEE/RSJ International Conference on Intelligent Robots and Systems (2025) 4
50. Sherrington, C.S.: The Integrative Action of the Nervous System. In: Scientific and Medical Knowledge Production, 1796-1918, pp. 217–253. Routledge (2023) 2
51. Siddiqui, Y., Frost, D., Aroudj, S., Avetisyan, A., Howard-Jenkins, H., DeTone, D., Moulon, P., Wu, Q., Li, Z., Straub, J., Newcombe, R., Engel, J.: ShapeR: Robust Conditional 3D Shape Generation from Casual Captures. In: CVPR (2026) 7
52. Todorov, E., Erez, T., Tassa, Y.: MuJoCo: A physics engine for model-based control. In: IEEE/RSJ International Conference on Intelligent Robots and Systems. pp. 5026–5033. IEEE (2012). <https://doi.org/10.1109/IR0S.2012.6386109> 20
53. Torne, M., Simeonov, A., Li, Z., Chan, A., Chen, T., Gupta, A., Agrawal, P.: Reconciling Reality Through Simulation: A Real-to-Sim-to-Real Approach for Robust Manipulation. In: RSS (2024) 2
54. Wang, H., Qiao, L., Jie, Z., Huang, Z., Feng, C., Zheng, Q., Ma, L., Lan, X., Liang, X.: X-SAM: From Segment Anything to Any Segmentation. In: AAAI (2026) 9
55. Wang, X., Zhou, B., Shi, Y., Chen, X., Zhao, Q., Xu, K.: Shape2Motion: Joint Analysis of Motion Parts and Attributes from 3D Shapes. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 8876–8884 (2019) 2
56. Weng, Y., Wen, B., Tremblay, J., Blukis, V., Fox, D., Guibas, L., Birchfield, S.: Neural Implicit Representation for Building Digital Twins of Unknown Articulated Objects. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) (2024) 2, 3
57. Werby, A., Buechner, M., Roefer, A., Huang, C., Burgard, W., Valada, A.: Articulated Object Estimation in the Wild. Conference on Robot Learning (CoRL) (2025) 3, 4, 10
58. Yuan, H., Li, X., Zhang, T., Sun, Y., Huang, Z., Xu, S., Ji, S., Tong, Y., Qi, L., Feng, J., Yang, M.H.: Sa2VA: Marrying SAM2 with LLaVA for Dense Grounded Understanding of Images and Videos. arXiv pre-print (2025) 9

59. Yuan, S., Shi, R., Wei, X., Zhang, X., Su, H., Liu, M.: LARM: A Large Articulated Object Reconstruction Model. In: Proceedings of the SIGGRAPH Asia 2025 Conference Papers (2025) 3
60. Zhang, C., Delitzas, A., Wang, F., Zhang, R., Ji, X., Pollefeys, M., Engelmann, F.: Open-Vocabulary Functional 3D Scene Graphs for Real-World Indoor Spaces. In: Proceedings of the Computer Vision and Pattern Recognition Conference. pp. 19401–19413 (2025) 2, 3, 4, 7
61. Zhang, K., Sha, S., Jiang, H., Loper, M., Song, H., Cai, G., Xu, Z., Hu, X., Zheng, C., Li, Y.: Real-to-Sim Robot Policy Evaluation with Gaussian Splatting Simulation of Soft-Body Interactions. In: ICRA (2026) 2

# Supplementary Material

## A Function Template Implementation Details

### A.1 Formalization Details

Here, we provide in more detail the formalization of the function template representation in terms of mappings between receptor part states and effector part states. In total, there are 4 possible combinations of state spaces, making up for 8 possible abstractions of mappings. However, we note that only 4 are commonly present in real-world scenarios. The below list describes each case.

1.  $\mathcal{M}_P : s_{\mathcal{R}} \rightarrow s_{\mathcal{E}}$ :

- (a)  $s_{\mathcal{R}}$  and  $s_{\mathcal{E}}$  are discrete finite sets. In this case, the sizes of  $s_{\mathcal{R}}$  and  $s_{\mathcal{E}}$  are usually equal. Thus, the mapping is a simple one-to-one mapping. For example, pressing the light switch turns the light on and off. We find that modeling the simplest case of each part admitting two states to be sufficient to cover our data distribution for discrete-to-discrete case, making the resulting mapping a **binary** function.
- (b)  $s_{\mathcal{R}}$  is a continuous set and  $s_{\mathcal{E}}$  is a discrete finite set. The most common function exemplar for this case from the real-world distribution is a **step** function. For example, opening the fridge door up to a certain angle triggers the interior lights to turn on.
- (c)  $s_{\mathcal{R}}$  and  $s_{\mathcal{E}}$  are continuous sets. In this case, there are many different valid functions between  $s_{\mathcal{R}}$  and  $s_{\mathcal{E}}$ . However, the real-world examples target simple control. Hence, in most cases,  $s_{\mathcal{R}}$  and  $s_{\mathcal{E}}$  change proportionally. We use a **linear** function to approximate this relationship. For example, controlling light intensity proportionally to the rotation of a knob.

2.  $\mathcal{M}_P : s_{\mathcal{R}} \times s_{\mathcal{E}} \rightarrow s_{\mathcal{E}}$ :

- (a)  $s_{\mathcal{R}}$  and  $s_{\mathcal{E}}$  are discrete finite sets. In this case,  $s_{\mathcal{R}}$  and  $s_{\mathcal{E}}$  are finite but not necessarily equinumerous. This case is common in many appliances such as washing machines, microwaves, and ovens which are controlled by a few buttons. Such transition functions can be arbitrarily complex. In this work, we consider one of the simplest forms – the **cumulative** function. This function maps change in  $s_{\mathcal{R}}$  to adding a certain value to  $s_{\mathcal{E}}$ . For example, pressing a “plus” button on an electric stove increases the temperature by 10 degrees.

Finally, the function template is defined as a function mapping between receptor and effector, which is decomposed to mapping and physical effect. These function templates can be further converted into Python scripts that are executable in specific simulators, such as Mujoco [52], Isaac Sim [40], or Genesis [2].

## A.2 Function Template Code Conversion Details

We first design a code template for each mapping:

– Binary:

```

1 def binary(receptor, effector):
2     if receptor.state:
3         effector.state = True # Or a real value
4     else:
5         effector.state = False # Or a real value
6

```

**Listing S1:** Pseudocode for binary function

– Step:

```

1 def step(receptor, effector):
2     if receptor.state > THRESHOLD:
3         effector.state = True # Or a real value
4     else:
5         effector.state = False # Or a real value
6

```

**Listing S2:** Pseudocode for step function

– Linear:

```

1 def linear(receptor, effector):
2     effector.state = C * receptor.state
3

```

**Listing S3:** Pseudocode for linear function

– Cumulative:

```

1 def cumulative(receptor, effector):
2     if receptor.state:
3         effector.state = effector.state + delta
4

```

**Listing S4:** Pseudocode for cumulative function

Then, the receptor and effector states are defined based on the target simulator. In our experiments, we demonstrated instantiation of geometry change in Isaac Sim [40], illumination change and temperature change in BEHAVIOR [28], and fluid change in Genesis [2].

To instantiate geometry change in Isaac Sim [40],  $s_{\mathcal{R}}$  and  $s_{\mathcal{E}}$  are the joint state: `joint_state=scene["Object"].data.joint_pos`. Joint states are changed by setting the target joint values: `scene["Object"].set_joint_position_target(torch.Tensor([[effector_target,receptor_target]]))`. We show a code example in Listing S5.

```

1 joint_state = scene["Microwave"].data.joint_pos # get
  receptor state
2 if joint_state[0][0] > 0.015: # check with THRESHOLD
3     scene["Microwave"].set_joint_position_target(torch.Tensor
      ([[np.pi / 2, receiver_target]])) # change effector state

```

**Listing S5:** Code example of a step function for geometry change in Isaac Sim

To instantiate illumination change in BEHAVIOR [28],  $s_{\mathcal{R}}$  is either `object_states.ToggledOn` for binary state or `object_states.Joint` for continuous state.  $s_{\mathcal{E}}$  is either `light_bulb.visible` for binary state or `light_bulb.intensity` for continuous state. The effector state  $s_{\mathcal{E}}$  is changed by simply assigning a boolean value or a real number, respectively. A code example is shown in Listing S6. To instantiate temperature change in BEHAVIOR,  $s_{\mathcal{E}}$  is `object_states.HeatSourceOrSink` and the computation of the temperature value is further defined in this class. A code example is shown in Listing S7. The full original source code for Listing S7 can be found on GitHub.

```

1 if light.states[object_states.ToggledOn].get_value():
2     light_bulb.visible = True
3 else:
4     light_bulb.visible = False

```

**Listing S6:** Code example of a binary function for illumination change in BEHAVIOR

```

1 class HeatSourceOrSink(AbsoluteObjectState,
  LinkBasedStateMixin, UpdateStateMixin):
2     ...
3     def _get_value(self):
4         # Check the toggle state.
5         if self.requires_toggled_on and not self.obj.states[
  ToggledOn].get_value():
6             return False
7
8         return True
9     ...

```

**Listing S7:** Code example of a binary function for temperature change in BEHAVIOR

To instantiate fluid change in Genesis [2], the receptor state  $s_{\mathcal{R}}$  is set to the joint value: `joint_state=faucet.get_dofs_position(receptor_idx)`. The effector state  $s_{\mathcal{E}}$  is set to `droplet_size` of the emitter, which is a real number. An example code snippet is shown in Listing S8.

Finally, we use simple strategies to compute a few parameter values needed in the function template. For `THRESHOLD` in the step mapping in Listing S2, we simply set it to be  $\text{THRESHOLD} = 0.7 \times \max(s_{\mathcal{R}})$ . For `C` is the linear mapping in Listing S3, we simply set it to be  $C = \frac{\max(s_{\mathcal{E}}) - \min(s_{\mathcal{E}})}{\max(s_{\mathcal{R}}) - \min(s_{\mathcal{R}})}$ . For the heat source position in temperature change and the light source position in illumination change, we set them to be the center of the bounding box of the effector mesh. Except for these parameters, we manually specify the `delta` in Listing S4, minimum

```

1 faucet_position = faucet.get_dofs_position(dofs_idx)[0]
2 droplet_size = change_rate * (faucet_position - joint_limits[
    dofs_idx[0], 0]) + MIN_DROPLET_SIZE
3 emitter.emit(
4     pos=emitter_position_recentered,
5     direction=np.array([0.0, 0.0, -1.0]),
6     speed=5,
7     droplet_shape="circle",
8     droplet_size=droplet_size,
9 )

```

**Listing S8:** Code example of a linear function for fluid change in Genesis

```

1 "This is a video. Analyze this video and answer the following questions: \
2 1. Which part of the object receives human action? \
3 2. Which part of the object reacts to human action? \
4 Please describe the name and features of the part as well as the spatial
5 relationship with surrounding objects. \
6 Please only answer in this template: \
7 {1: {name: xxx, description: aaa}, 2: {name: yyy, description: bbb}} \
8 Substitute \"xxx\" and \"yyy\" with the name of the part of the object, \"aaa
9 \" and \"bbb\" with the description of the part. \
10 DO NOT answer any other information."

```

**Listing S9:** Text prompt for receptor and effector identification and description

and maximum temperature for temperature change, illumination intensity for illumination change, and droplet size for fluid change. We also manually set the emitter position in fluid change.

## B Additional Experimental Details

### B.1 Receptor and Effector Descriptions

We use Gemini 3 Flash [17] for initial video understanding to infer the descriptions of the receptor part and the effector part. We provide the whole video as input, along with a prompt that specifies to identify the part human interacts with and the part that exhibits a reaction to it, as well as a description of spatial relationships of these parts. The exact prompt can be found in Listing S9.

### B.2 2D Segmentation

**Implementation details.** As the methods we choose rely on a combination of VLM and SAM models, they frequently support a loop where they prompt SAM, evaluate the outputs, refine the prompt and repeat. We allow up to 10 such iterations per frame. We deem this necessary, as the egocentric videos from our dataset are very challenging to segment, while SAM might need multiple rounds of prompt refinement to steer properly. This results in a setup like SAM3 & Qwen3VL taking a lot of time, as we opt for SAM3 Agent setup from the official

**Table S1:** Evaluation of 2D segmentation performance in the oracle setting, with ground truth receptor and effector labels provided.

| Model          | IoU (%)     |             |             | Success (%) |             | Avg. Run. (s) |
|----------------|-------------|-------------|-------------|-------------|-------------|---------------|
|                | Receptor    | Effector    | Avg.        | Receptor    | Effector    |               |
| VisionReasoner | 13.1        | 47.4        | 30.2        | 6.3         | 48.0        | 437           |
| SAM3 & Qwen3VL | <b>30.2</b> | <b>61.7</b> | <b>46.0</b> | <b>20.2</b> | <b>75.3</b> | 1950          |
| SAM3 & Molmo2  | 12.2        | 39.9        | 26.0        | 7.0         | 31.0        | 419           |
| X-SAM          | 1.9         | 12.6        | 7.2         | 0.4         | 8.5         | 336           |
| SA2VA          | 14.8        | 43.4        | 29.1        | 1.4         | 33.6        | 1956          |

SAM3 codebase which involves a particularly long system prompt defining a variety of tool calls a VLM can perform.

**Performance in oracle setting.** Additionally, we consider an oracle setting where the ground truth labels for receptor and effector are given. Note that in this setup, longer, spatially-enhanced descriptions of the parts are not available. We find that the trends of relative performance between the selected baselines hold as can be seen in Tab. S1. Ground truth labels lead to generally better performance for the effectors. For the receptors, the performance generally decreases, due to lack of longer descriptions with ground truth labels. Without such spatially-enhanced descriptions, the methods struggle to disambiguate the desired instance in the cases of multiple instances of the same part present in the frame.

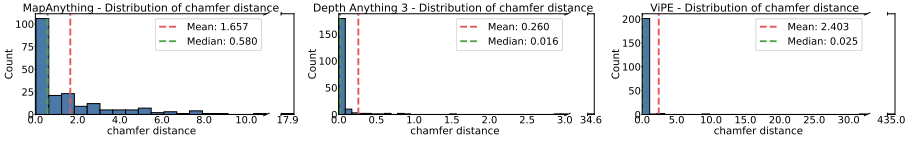
### B.3 Reconstruction

**Implementation details.** Due to CUDA memory limits, we divide the input video into several chunks when benchmarking MapAnything [25] and Depth Anything 3 [30]. To stitch reconstruction results for each chunk together, we let the first input video frame of the current video chunk be the same as the last input video frame of the previous video chunk. After reconstructing each chunk, we use the predicted camera pose of the last video frame of the first chunk and the predicted camera pose of the first video frame of the second chunk to compute the transformation. We sequentially transform the reconstruction results of each chunk into the same coordinate system. In practice, we set the length of each video chunk to be 20 frames.

**On the choice of median over mean Chamfer Distance.** We plot the Chamfer Distance distribution on different methods, with video frames and predicted part masks as input in Fig. S1. We can see that there are a few extreme values in the reconstruction results, and therefore the mean values are skewed. We also draw the median values and mean values on the charts. We can see that median values lie in the range with most results. Therefore, using the median value can better reflect the overall performance of reconstruction methods.

**Ablation study.** We conduct reconstruction experiments across different input modalities, as shown in Tab. S2 and Fig. S2. Comparing experiments with and





**Fig. S1:** Chamfer Distance distribution on three different methods with RGB video frames and predicted part masks as input. We can see that there are a few extreme values in the reconstruction results. Thus, the median value can better reflect the performance of different reconstruction methods.

**Table S2:** Evaluating 4D reconstruction. For inputs including ground truth masks, we report the median value of the chamfer distance across all videos. Among them, Depth Anything 3 performs the best. We also find that the fusion module does not help with reconstruction.

| Input Part Masks   | Input Camera | Method                      | Receptor CD (m <sup>2</sup> ) | Effector CD (m <sup>2</sup> ) | Total CD (m <sup>2</sup> ) | Failure Rate (%) |
|--------------------|--------------|-----------------------------|-------------------------------|-------------------------------|----------------------------|------------------|
| Ground Truth Masks | ✓            | MapAnything                 | 0.022                         | 0.013                         | 0.017                      | 0                |
|                    |              | MapAnything w/o fusion      | 0.021                         | 0.012                         | 0.016                      | 0                |
|                    |              | Depth Anything 3            | <b>0.006</b>                  | 0.009                         | <b>0.007</b>               | 5.16             |
|                    |              | Depth Anything 3 w/o fusion | <b>0.006</b>                  | <b>0.007</b>                  | <b>0.007</b>               | 5.16             |
| Predicted Masks    | ✓            | MapAnything                 | 0.060                         | 0.033                         | 0.043                      | 0                |
|                    |              | Depth Anything 3            | <b>0.033</b>                  | <b>0.015</b>                  | <b>0.016</b>               | 4.830            |
| Ground Truth Mask  | ✗            | MapAnything                 | 0.536                         | 0.530                         | 0.535                      | 0                |
|                    |              | MapAnything w/o fusion      | 0.535                         | 0.467                         | 0.505                      | 0                |
|                    |              | Depth Anything 3            | <b>0.004</b>                  | <b>0.006</b>                  | <b>0.005</b>               | 0                |
|                    |              | Depth Anything 3 w/o fusion | <b>0.004</b>                  | <b>0.006</b>                  | <b>0.005</b>               | 0                |
|                    |              | ViPE                        | 0.009                         | 0.009                         | 0.009                      | 0                |
|                    |              | ViPE w/o fusion             | 0.009                         | 0.008                         | 0.009                      | 0                |
| Predicted Mask     | ✗            | MapAnything                 | 0.380                         | 0.953                         | 0.580                      | 0                |
|                    |              | Depth Anything 3            | <b>0.026</b>                  | <b>0.014</b>                  | <b>0.016</b>               | 0                |
|                    |              | ViPE                        | 0.034                         | 0.021                         | 0.025                      | 0                |

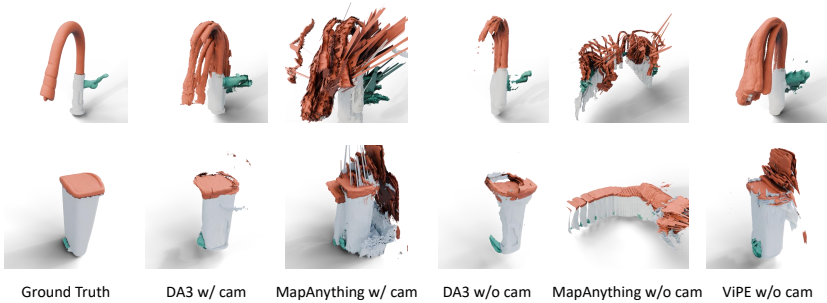
without ground truth camera parameters input, we find that MapAnything requires accurate camera parameters to achieve high reconstruction accuracy. We also examine the effects of the fusion module. Comparing results between the method with and without the fusion module, we find that the fusion module does not help with reconstruction. We hypothesize that this is due to two reasons: 1) object parts are usually too small to provide reliable feature matching to compute transformations; and 2) many parts are textureless or reflective, increasing the difficulty of finding reliable feature matches.

## B.4 Articulation Estimation

We conduct articulation estimation experiments across different input modalities, as shown in Tab. S3. We observe that given ground-truth part masks, performance improves only marginally.

## B.5 Function Template Prediction

We evaluate function template prediction with ground truth part masks in Tab. S4.



**Fig. S2:** Example results for reconstruction using ground truth part masks. MapAnything exhibits severe drifting issues as predicted camera poses for different video frames are inaccurate.

**Table S3:** Evaluating articulation parameter estimation. We can find that the overall performance for both methods is very inaccurate, indicating that articulation estimation is the bottleneck of this task and the whole system.

| Input Part Masks   | Method    | Joint Axis Err. (rad) ↓ | Joint Origin Err. (m) ↓ | Joint Type Acc. (%) ↑ | Failure Rate (%) ↓ |
|--------------------|-----------|-------------------------|-------------------------|-----------------------|--------------------|
| Ground Truth Masks | ArtiPoint | 1.120                   | <b>0.383</b>            | <b>80.769</b>         | 68.651             |
|                    | iTACO     | <b>1.003</b>            | 0.603                   | 44.262                | <b>11.111</b>      |
| Predicted Masks    | ArtiPoint | 1.057                   | <b>0.346</b>            | <b>74.211</b>         | 46.479             |
|                    | iTACO     | <b>1.022</b>            | 0.665                   | 26.866                | <b>5.634</b>       |

## B.6 Prompting Details for Function Prediction

We prompt VLMs with both text and video for function prediction. The text prompt is shown in Listing S10. An example video frame annotated in the way required for function prediction is shown in Fig. S3.

## B.7 Final Results in Simulation

We show additional final outputs from our baseline approach in Fig. S4.

## B.8 Additional Discussion of Limitations

We currently only consider simple functionalities with one receptor and one effector. In reality, there could be multiple receptors and multiple effectors in one part functionality. Also, in Appendix A.1, we do not cover all possible combinations as we did not observe any real part functionalities for some of the cases in our dataset. However, this does not mean such part functionalities do not exist in other real-world scenarios.

Another limitation is that our current function template implementation still requires manual specification of some parameter values, such as the particle emitter positions for fluid physical effects.

**Table S4:** Evaluating function understanding. We report the accuracy of each aspect and the union of them. Overall predictions are counted as correct when two aspects are predicted correctly. Among the four different VLM, Gemini-3-flash performs the best.

| Input Part Masks   | Method              | Physical Effect Acc. (%) $\uparrow$ | Mapping Acc. (%) $\uparrow$ | Overall Acc. (%) $\uparrow$ |
|--------------------|---------------------|-------------------------------------|-----------------------------|-----------------------------|
| Ground Truth Masks | Gemini-3-flash      | <b>95.572</b>                       | <b>88.192</b>               | <b>85.977</b>               |
|                    | GPT-5-mini          | 87.454                              | 81.550                      | 75.276                      |
|                    | Molmo-2-8B          | 81.550                              | 31.735                      | 28.044                      |
|                    | Qwen3VL-8B-Instruct | 86.716                              | 71.218                      | 67.158                      |
| Predicted Masks    | Gemini-3-flash      | 95.238                              | <b>97.619</b>               | <b>92.857</b>               |
|                    | GPT-5-mini          | 88.095                              | 90.476                      | 83.333                      |
|                    | Molmo-2-8B          | 90.476                              | 30.952                      | 28.571                      |
|                    | Qwen3VL-8B-Instruct | <b>97.619</b>                       | 76.190                      | 76.190                      |



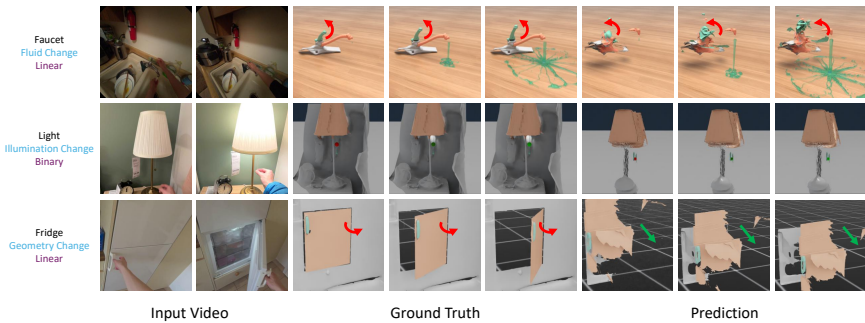
**Fig. S3:** Example video frame we use to prompt VLM for function prediction. The part marked in green is the receptor, and the part marked in red is the effector.

```

1 "This is a video. The receptor part marked in green and the effector part
  marked in red have functional relationship. Function has two attributes:
  physical effect and numerical relationship. \
2 Physical effect refers to the physical phenomenon of state change of the
  effector. We consider four different physical effects: geometry change,
  illumination change, temperature change, and fluid change. \
3 If the function results in the change of shape or position of the effector,
  the physical effect is geometry change. \
4 If the function results in the change of illumination status of the effector,
  such as brightness or color, the physical effect is illumination change. \
5 If the function results in the change of temperature status of the effector,
  such as becoming hotter or colder, the physical effect is temperature change.
  \
6 If the function results in the change of fluid status of the effector, such
  as fluid flowing in or out, the physical effect is fluid change. \
7 Numerical relationship refers to the mathematical relationship between the
  state of the receiver and the state of the effector. We consider four
  different numerical function: binary function, step function, linear function
  , cumulative function. \
8 If both the states of the receptor and effector have only two possible values
  , and they are mapped one-to-one, the numerical relationship is binary
  function. \
9 For example, the light switch is a binary function, where the state of the
  receptor (the switch) can be either on or off, and the state of the effector
  (the light) can also be either on or off. \
10 If the effector state changes only when the receiver state continuously
  changes until a certain threshold is reached, the numerical relationship is
  step function. \
11 For example, when pressing the button of a microwave to a certain distance,
  the microwave's door will open. The state of the receptor (the button)
  continuously changes as you press it, but the state of the effector (the
  microwave door) only changes when the button is pressed to a certain distance
  . \
12 If both states change continuously in a linear manner, the numerical
  relationship is linear function. \
13 For example, the volume of water flowing out of a faucet is a linear function
  of how much you turn the faucet. The state of the receptor (the faucet) and
  the state of the effector (the water flow) both change continuously, and they
  have a linear relationship. \
14 If the effector state accumulates over time as the receiver state changes,
  the numerical relationship is cumulative function. \
15 For example, when you press \"increase temperature\" button of a electric
  induction cooker, the temperature of the cooker will increase a certain value
  . The state of the receptor (the button) changes as you keep pressing it, and
  the state of the effector (the cooker temperature) accumulates over time. \
16 Analyze this video and answer the following questions: \
17 1. Which physical effect best describes the functional relationship between
  the red part and green part? Please choose one from the following: \
18 (a) geometry change \
19 (b) illumination change \
20 (c) temperature change \
21 (d) fluid change \
22 2. Which function best describes the numerical relationship between the state
  of the red part and the green part? Please choose one from the following: \
23 (a) binary function \
24 (b) step function \
25 (c) linear function \
26 (d) cumulative function \
27 Please only answer in this template: \
28 {\"i\": \"xxx\", \"2\": \"yyy\", \"reason\": \"zzz\"} \
29 Substitue \"xxx\" and \"yyy\" with the option letter a / b / c / d for each
  question. Also tell me the reason of choosing each option by substituting \"
  zzz\". \
30 DO NOT answer any other information."

```

**Listing S10:** Text prompt for function prediction



**Fig. S4:** Additional qualitative results of the final outputs of our system. The first row illustrates an interactive faucet in Genesis [2]. The second row illustrates an interactive lamp in BEHAVIOR-1K [28]. The third row illustrates an interactive fridge door in Isaac Sim [40]. We use teal to indicate receptors and orange to indicate effectors. Red and green arrows represent revolute and prismatic joints respectively.